

- 2) 当 $n > 1$ 时, 其余结点可分为 m ($m > 0$) 个互不相交的有限集 $T_1, T_2, \dots, T_m$, 其中每个集合本身又是一棵树, 称为根的子树。

显然, 树的定义是递归的——在定义中引用了自身, 因此树是一种典型的递归数据结构。作为一种逻辑结构, 树同时也是一种分层结构, 具有以下两个特征:

- 1) 根结点没有前驱, 除根结点外的每个结点有且仅有一个前驱。
- 2) 所有结点均可拥有零个或多个后继。

树适用于表示具有层次结构的数据。除根结点外, 树中任一结点最多只与上一层的一个结点(其父结点)存在直接关联; 根结点则无上层结点。因此, 在包含 n 个结点的树中, 恰好存在 $n-1$ 条边。同时, 每个结点可与其下一层的零个或多个结点(其孩子结点)建立直接联系。

5.1.2 基本术语

下面结合图 5.1 中的树来说明一些基本术语和概念。

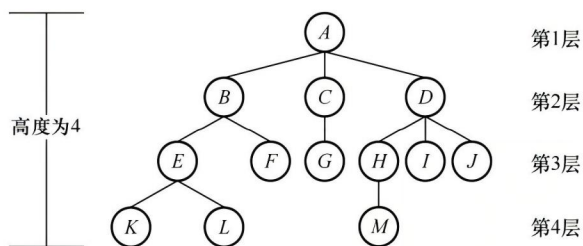


图 5.1 树的树形表示

- 1) 祖先、子孙、双亲、孩子、兄弟和堂兄弟。

考虑结点 K , 从根 A 到结点 K 的唯一路径上所经过的所有其他结点, 称为 K 的祖先。例如, 结点 B 是 K 的祖先, 而 K 是 B 的子孙; B 的子孙包括 E, F, K, L 。该路径上最接近 K 的结点 E 称为 K 的双亲, K 则是 E 的孩子。根结点 A 是树中唯一没有双亲的结点。具有相同双亲的结点互为兄弟, 如 K 与 L 的双亲均为 E , 故 K 与 L 为兄弟。若两个结点的双亲位于同一层, 则它们互为堂兄弟, 如 G 与 E, F, H, I, J 互为堂兄弟。

- 2) 结点的层次、深度和高度。

结点的层次从根开始定义: 根为第 1 层, 其孩子为第 2 层, 以此类推。结点的深度即为其所在的层次。树的高度(或深度)是树中结点的最大层数。而结点的高度是指以其为根的子树的高度。图 5.1 中树的高度为 4。

- 3) 结点的度和树的度。

树中一个结点的孩子个数称为该结点的度; 树中所有结点的度的最大值称为树的度。例如, 结点 B 的度为 2, 结点 D 的度为 3, 因此该树的度为 3。

- 4) 分支结点和叶结点。

度大于 0 的结点称为分支结点(也称非终端结点); 度为 0(无孩子)的结点称为叶结点(也称终端结点)。在分支结点中, 每个结点的分支数即为其度。

- 5) 有序树和无序树。

若树中各结点的子树从左到右具有固定次序、不可互换, 则称为有序树; 否则称为无序树。假设图 5.1 为有序树, 若交换某结点的子结点位置, 则得到一棵不同的树。

- 6) 路径和路径长度。

树中两个结点之间的路径是由从一个结点到另一个结点所经过的结点序列构成的, 路径长度则是该路径上边的数目。

04. 对于一棵具有 n 个结点、度为 4 的树来说, ()。
- A. 树的高度至多是 $n-3$ B. 树的高度至多是 $n-4$
C. 第 i 层上至多有 $4(i-1)$ 个结点 D. 至少在某一层上正好有 4 个结点
05. 度为 4、高度为 h 的树, ()。
- A. 至少有 $h+3$ 个结点 B. 至多有 $4h-1$ 个结点
C. 至多有 $4h$ 个结点 D. 至少有 $h+4$ 个结点
06. 假定一棵度为 3 的树中, 结点数为 50, 则其最小高度为 ()。
- A. 3 B. 4
C. 5 D. 6
07. 设有一棵度为 3 的树, 其中度为 3 的结点数 $n_3 = 2$, 度为 2 的结点数 $n_2 = 1$, 叶结点数 $n_0 = 6$, 则该树的结点总数为 ()。
- A. 12 B. 9
C. 10 D. ≥ 9 的任意整数
08. 设一棵 m 叉树中有 N_1 个度数为 1 的结点, N_2 个度数为 2 的结点…… N_m 个度数为 m 的结点, 则该树中共有 () 个叶结点。
- A. $\sum_{i=1}^m (i-1)N_i$ B. $\sum_{i=1}^m N_i$ C. $\sum_{i=2}^m (i-1)N_i$ D. $\sum_{i=2}^m (i-1)N_i + 1$
09. 【2010 统考真题】在一棵度为 4 的树 T 中, 若有 20 个度为 4 的结点, 10 个度为 3 的结点, 1 个度为 2 的结点, 10 个度为 1 的结点, 则树 T 的叶结点数是 ()。
- A. 41 B. 82 C. 113 D. 122
10. 【2016 统考真题】若森林 F 有 15 条边、25 个结点, 则 F 包含树的个数是 ()。
- A. 8 B. 9 C. 10 D. 11

二、综合应用题

01. 含有 n 个结点的三叉树的最小高度是多少?
02. 已知一棵度为 4 的树中, 度为 0, 1, 2, 3 的结点数分别为 14, 4, 3, 2, 求该树的结点总数 n 和度为 4 的结点数, 并给出推导过程。
03. 已知一棵度为 m 的树中, 有 n_1 个度为 1 的结点, 有 n_2 个度为 2 的结点……有 n_m 个度为 m 的结点, 问该树有多少个叶结点?

5.1.5 答案与解析

一、单项选择题

01. D

树是一种分层结构, 它特别适合组织那些具有分支层次关系的数据。

02. A

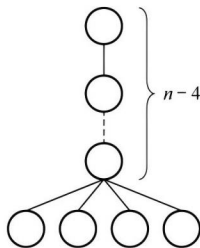
除根结点外, 其他每个结点都是某个结点的孩子, 因此树中所有结点的度数加 1 等于结点数, 即所有结点的度数之和等于总结点数减 1。这是一个重要的结论, 做题时经常用到。

03. A

树的路径长度是指树根到每个结点的路径长的总和, 根到每个结点的路径长度的最大值应是树的高度减 1。注意与哈夫曼树的带权路径长度相区别。

04. A

要使得具有 n 个结点、度为 4 的树的高度最大, 就要使得每层的结点数尽可能少, 类似右图所示的树, 除最后一层外, 每层的结点数是 1, 最终该树的高度为 $n-3$ 。树的度为 4 只能说明存在某结点正好 (也最多) 有 4 个孩子结点, 选项 D 错误。

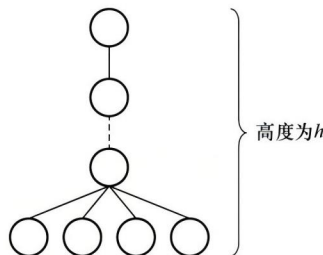


05. A

要使得度为 4、高度为 h 的树的总结点数最少, 需要满足以下两个条件:

- ① 至少有一个结点有 4 个分支。
- ② 每层的结点数尽可能少。

情况类似右图所示的树, 结点数为 $h+3$ 。



要使得度为 4、高度为 h 的树的总结点数最多, 应使每个非叶结点的度均为 4, 即满树, 总结点数最多为 $1+4+4^2+\dots+4^{h-1}$ 。

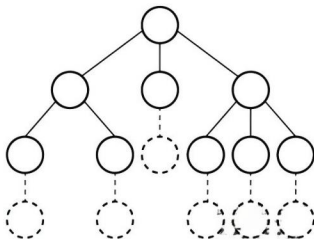
对于上面的两题, 应画出草图来求解, 就能一目了然。

06. C

要求满足条件的树, 那么该树是一棵完全三叉树。在度为 3 的完全三叉树中, 第 1 层有 1 个结点, 第 2 层有 $3^1=3$ 个结点, 第 3 层有 $3^2=9$ 个结点, 第 4 层有 $3^3=27$ 个结点, 因此结点数之和为 $1+3+9+27=40$, 第 5 层的结点数 $=50-40=10$ 个, 因此最小高度为 5。

07. D

总结点数 $n=n_0+n_1+n_2+n_3=6+n_1+1+2=n_1+9$, 总度数 $=n-1=n_1+2n_2+3n_3=n_1+2+6=n_1+8$, 根据题目条件无法得出 n 的具体值, 只能证明 n 是一个大于或等于 9 的任意整数。画出满足题目条件的树, 可以是右图所示的一棵树, 该树中无法确定 n 的具体数量。



08. D

设叶结点数为 N_0 , 总结点数为 N , 则 $N=N_1+2N_2+3N_3+\dots+mN_m+1$, 又因为 $N=N_0+N_1+N_2+N_3+\dots+N_m$, 所以 $N_0=N_2+2N_3+\dots+(m-1)N_m+1=\sum_{i=2}^m (i-1)N_i+1$ 。

09. B

设树中度为 i ($i=0, 1, 2, 3, 4$) 的结点数分别为 n_i , 树中结点总数为 n , 则 $n = \text{分支数} + 1$, 而分支数又等于树中各结点的度之和, 即 $n = 1 + n_1 + 2n_2 + 3n_3 + 4n_4 = n_0 + n_1 + n_2 + n_3 + n_4$ 。依题意, $n_1 + 2n_2 + 3n_3 + 4n_4 = 10 + 2 + 30 + 80 = 122$, $n_1 + n_2 + n_3 + n_4 = 10 + 1 + 10 + 20 = 41$, 可得出 $n_0 = 82$, 即树 T 的叶结点的个数是 82。

10. C

解法1: 树有一个重要性质, 即在 n 个结点的树中有 $n-1$ 条边, “那么对于每棵树, 其结点数比边数多 1”。本题森林中的结点数比边数多 10 ($25-15=10$), 显然共有 10 棵树。

解法2: 仔细分析后发现此题也是考查图的性质: 生成树和生成森林。对于图的生成树有一个重要的性质, 即图中顶点数若为 n , 则其生成树含有 $n-1$ 条边。对比解法1中树的性质, 不难发现两种解法都用到了性质 “树中结点数比边数多 1”, 后面的分析如解法1。

映结点之间的逻辑关系,既能最大限度地节省存储空间,又可直接通过数组下标快速确定结点在二叉树中的位置及其父子、兄弟关系。

考点追踪 ▶▶ 二叉树的顺序存储结构相关应用 (2020、2025)

然而,对于一般的二叉树,若仍想通过数组下标反映逻辑结构,则需插入若干“空结点”,使其结构与同高度的完全二叉树一致,再将所有结点(包括空结点)按层序存入数组,这种做法在最坏情况下效率极低。例如,一个高度为 h 且仅有 h 个结点的单支树,却需要占用近 $2^h - 1$ 个存储单元。图 5.4 展示了二叉树的顺序存储结构,其中用 0 表示空结点。

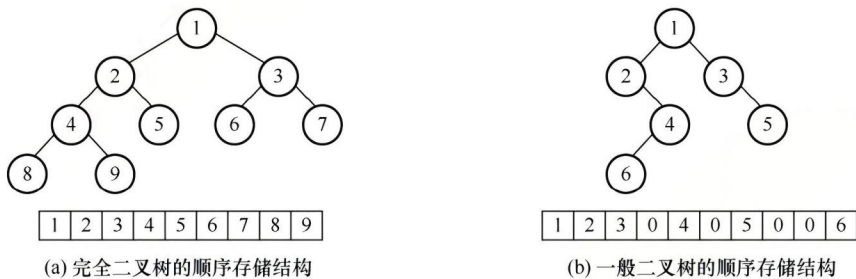


图 5.4 二叉树的顺序存储结构

注意

建议从数组下标 1 开始存储树中结点,保证数组下标与结点编号一致。

2. 链式存储结构

由于顺序存储的空间利用率较低,二叉树通常采用链式存储结构,即使用链表结点来存储二叉树中的每个结点。在二叉树的链式存储中,每个结点通常包括若干数据域和指针域。二叉链表至少包含 3 个域:左指针域 lchild、数据域 data 和右指针域 rchild,如图 5.5 所示。



图 5.5 二叉树链式存储的结点结构

图 5.6 展示了一棵二叉树及其对应的二叉链表。在实际应用中,还可根据需要扩展结点结构,例如增加指向父结点的指针域,从而形成三叉链表的存储结构。

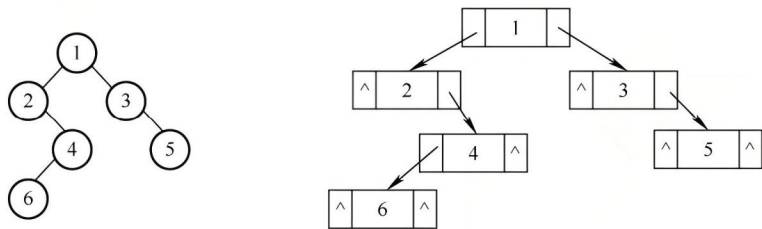


图 5.6 二叉链表的存储结构

二叉树的链式存储结构可定义如下:

```
typedef struct BiTNode{
    ElemType data;           //数据域
    struct BiTNode *lchild,*rchild; //分别指向左右孩子的指针
}BiTNode,*BiTree;
```

不同的存储结构会导致二叉树操作算法的实现不同,因此应根据实际应用场景(如二叉树的

形态特征及所需执行的运算类型)选择合适的存储方式。

容易验证,在含有 n 个结点的二叉链表中,共有 $n+1$ 个空链域(重要结论),常出现在选择题中。下一节将利用这些空链域构造一种新的链表结构——线索链表。

5.2.3 本节试题精选

一、单项选择题

01. 下列关于二叉树的说法中,正确的是()。
 - A. 度为2的有序树就是二叉树
 - B. 含有 n 个结点的二叉树的高度为 $\lfloor \log_2 n \rfloor + 1$
 - C. 在完全二叉树中,若一个结点没有左孩子,则它必是叶结点
 - D. 含有 n 个结点的完全二叉树的高度为 $\lceil \log_2 n \rceil$
02. “二叉树为空”意味着二叉树()。
 - A. 根结点没有子树
 - B. 不存在
 - C. 没有结点
 - D. 由一些没有赋值的空结点构成
03. 下列关于完全二叉树的说法中,正确的是()。
 - A. 在完全二叉树中,叶结点的双亲的左兄弟(若存在)一定不是叶结点
 - B. 任何一棵二叉树中,叶结点数为度为2的结点数减1,即 $n_0 = n_2 - 1$
 - C. 完全二叉树不适合顺序存储结构,只有满二叉树适合顺序存储结构
 - D. 结点按完全二叉树层序编号的二叉树中,第 i 个结点的左孩子的编号为 $2i$
04. 具有10个叶结点的二叉树中有()个度为2的结点。
 - A. 8
 - B. 9
 - C. 10
 - D. 11
05. 设高度为 h 的二叉树上只有度为0和度为2的结点,则此类二叉树中所包含的结点数至少为()。
 - A. h
 - B. $2h-1$
 - C. $2h+1$
 - D. $h+1$
06. 具有 n 个结点且高度为 n 的二叉树的数目为()。
 - A. $\log_2 n$
 - B. $n/2$
 - C. n
 - D. 2^{n-1}
07. 假设一棵二叉树的结点数为50,则它的最小高度是()。
 - A. 4
 - B. 5
 - C. 6
 - D. 7
08. 设二叉树有 $2n$ 个结点,且 $m < n$,则不可能存在()的结点。
 - A. n 个度为0
 - B. $2m$ 个度为0
 - C. $2m$ 个度为1
 - D. $2m$ 个度为2
09. 一个具有1025个结点的二叉树的高 h 为()。
 - A. 11
 - B. 10
 - C. $11 \sim 1025$
 - D. $10 \sim 1024$
10. 设二叉树只有度为0和2的结点,其结点数为15,则该二叉树的最大深度为()。
 - A. 4
 - B. 5
 - C. 8
 - D. 9
11. 高度为 h 的完全二叉树最少有()个结点。
 - A. 2^h
 - B. $2^h + 1$
 - C. 2^{h-1}
 - D. $2^h - 1$
12. 已知一棵完全二叉树的第6层(设根为第1层)有8个叶结点,则完全二叉树的结点数最少是()。
 - A. 39
 - B. 52
 - C. 111
 - D. 119
13. 若一棵深度为6的完全二叉树的第6层有3个叶结点,则该二叉树共有()个叶结点。
 - A. 17
 - B. 18
 - C. 19
 - D. 20

14. 一棵完全二叉树上有 1001 个结点, 其中叶结点的个数是 ()。
- A. 250 B. 500 C. 254 D. 501
15. 若一棵二叉树有 126 个结点, 在第 7 层 (根结点在第 1 层) 至多有 () 个结点。
- A. 32 B. 64 C. 63 D. 不存在第 7 层
16. 一棵有 124 个叶结点的完全二叉树, 最多有 () 个结点。
- A. 247 B. 248 C. 249 D. 250
17. 某完全二叉树 T 中, 结点数最大的层有 8 个结点, 则 T 中至多有 () 个结点。
- A. 8 B. 15 C. 23 D. 31
18. 一棵有 n 个结点的二叉树采用二叉链存储结点, 其中空指针数为 ()。
- A. n B. $n+1$ C. $n-1$ D. $2n$
19. 设有 n ($n \geq 1$) 个结点的二叉树采用三叉链表表示, 其中每个结点包含三个指针, 分别指向其左孩子、右孩子及双亲 (若不存在, 则置为空), 则下列说法中正确的是 ()。
- I. 树中空指针的数量为 $n+2$ II. 所有度为 2 的结点均被三个指针指向
III. 每个叶结点均被一个指针所指向
- A. I B. I、II C. I、III D. II、III
20. 在一棵完全二叉树中, 其根的序号为 1, () 可判定序号为 p 和 q 的两个结点是否在同一层。
- A. $\lfloor \log_2 p \rfloor = \lfloor \log_2 q \rfloor$ B. $\log_2 p = \log_2 q$
C. $\lfloor \log_2 p \rfloor + 1 = \lfloor \log_2 q \rfloor$ D. $\lfloor \log_2 p \rfloor = \lfloor \log_2 q \rfloor + 1$
21. 在一个用数组表示的完全二叉树中, 根结点的下标为 1, 那么下标为 17 和 19 的结点的最近公共祖先的下标是 ()。
- A. 1 B. 2 C. 4 D. 8
22. 假定一棵三叉树的结点数为 50, 则它的最小高度为 ()。
- A. 3 B. 4 C. 5 D. 6
23. 具有 n 个结点的三叉树用三叉链表表示, 则树中空指针域的个数为 ()。
- A. $3n+1$ B. $2n+1$ C. $3n-1$ D. $3n$
24. 对于一棵满二叉树, 共有 n 个结点和 m 个叶结点, 高度为 h , 则 ()。
- A. $n = h + m$ B. $n + m = 2h$ C. $m = h - 1$ D. $n = 2^h - 1$
25. 【2009 统考真题】已知一棵完全二叉树的第 6 层 (设根为第 1 层) 有 8 个叶结点, 则该完全二叉树的结点数最多是 ()。
- A. 39 B. 52 C. 111 D. 119
26. 【2011 统考真题】若一棵完全二叉树有 768 个结点, 则该二叉树中叶结点的个数是 ()。
- A. 257 B. 258 C. 384 D. 385
27. 【2018 统考真题】设一棵非空完全二叉树 T 的所有叶结点均位于同一层, 且每个非叶结点都有 2 个子结点。若 T 有 k 个叶结点, 则 T 的结点总数是 ()。
- A. $2k-1$ B. $2k$ C. k^2 D. 2^k-1
28. 【2020 统考真题】对于任意一棵高度为 5 且有 10 个结点的二叉树, 若采用顺序存储结构保存, 每个结点占 1 个存储单元 (仅存放结点的数据信息), 则存放该二叉树需要的存储单元数量至少是 ()。
- A. 31 B. 16 C. 15 D. 10
29. 【2022 统考真题】若三叉树 T 中有 244 个结点 (叶结点的高度为 1), 则 T 的高度至少是 ()。

A. 8

B. 7

C. 6

D. 5

30. 【2025 统考真题】若二叉树的结点值均为正整数, 采用顺序存储方式保存在数组 R 中, 用 -1 表示结点不存在, 则下列数组中, 不能表示一棵二叉树的是 ()。

A. $R[] = \{20, 15, 40, -1, -1, 35\}$ B. $R[] = \{15, 40, 10, 18, 35, -1, -1\}$ C. $R[] = \{15, 40, 10, -1, -1, -1, 12\}$ D. $R[] = \{17, 20, 35, -1, 18, 45, -1, -1, 19, 27\}$

二、综合应用题

01. 在一棵完全二叉树中, 含有 n_0 个叶结点, 当度为 1 的结点数为 1 时, 该树的高度是多少? 当度为 1 的结点数为 0 时, 该树的高度是多少?

02. 一棵有 n 个结点的满二叉树有多少个分支结点和多少个叶结点? 该满二叉树的高度是多少?

03. 已知完全二叉树的第 9 层有 240 个结点, 则整个完全二叉树有多少个结点? 有多少个叶结点?

04. 一棵高度为 h 的满 m 叉树有如下性质: 根结点所在层次为第 1 层, 第 h 层上的结点都是叶结点, 其余各层上每个结点都有 m 棵非空子树, 若按层次自顶向下, 同一层自左向右, 顺序从 1 开始对全部结点进行编号, 试问:

1) 各层的结点数是多少?

2) 编号为 i 的结点的双亲结点 (若存在) 的编号是多少?

3) 编号为 i 的结点的第 k 个孩子结点 (若存在) 的编号是多少?

4) 编号为 i 的结点有右兄弟的条件是什么? 其右兄弟结点的编号是多少?

05. 已知一棵二叉树按顺序存储结构进行存储, 设计一个算法, 求编号分别为 i 和 j 的两个结点的最近的公共祖先结点的值。

06. 【2016 统考真题】若一棵非空 k ($k \geq 2$) 叉树 T 中的每个非叶结点都有 k 个孩子, 则称 T 为正则 k 叉树。请回答下列问题并给出推导过程。

1) 若 T 有 m 个非叶结点, 则 T 中的叶结点有多少个?

2) 若 T 的高度为 h (单结点的树 $h=1$), 则 T 的结点数最多为多少个? 最少为多少个?

5.2.4 答案与解析

一、单项选择题

01. C

在二叉树中, 若某个结点只有一个孩子, 则这个孩子的左右次序是确定的; 而在度为 2 的有序树中, 若某个结点只有一个孩子, 则这个孩子就无须区分其左右次序, 选项 A 错误。选项 B 仅当是完全二叉树时才有意义, 对于任意一棵二叉树, 高度可能为 $\lfloor \log_2 n \rfloor + 1 \sim n$ 。在完全二叉树中, 若有度为 1 的结点, 则只可能有一个, 且该结点只有左孩子而无右孩子, 选项 C 正确。完全二叉树的高度为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$, 也可通过举例 $n=4$ 来排除, 选项 D 错误。

02. C

“二叉树为空”意味着二叉树中没有结点, 但并不意味着二叉树不存在。注意, 线性表可以是空表, 树可以是空树, 但图不能是空图 (图中不能没有结点)。

03. A

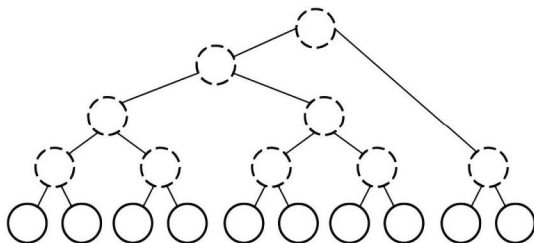
在完全二叉树中, 叶结点的双亲的左兄弟的孩子一定在其前面 (且一定存在), 所以双亲的左兄弟 (若存在) 一定不是叶结点, 选项 A 正确。 n_0 应等于 $n_2 + 1$, 选项 B 错误。完全二叉树和满二叉树均可以采用顺序存储结构, 选项 C 错误。第 i 个结点的左孩子不一定存在, 选项 D 错误。

选项 B 的这种通用公式适用于所有二叉树，我们应能立即联想到采用特殊值代入法验证，如画一个只含 3 个结点的满二叉树的草图来验证是否满足条件。

04. B

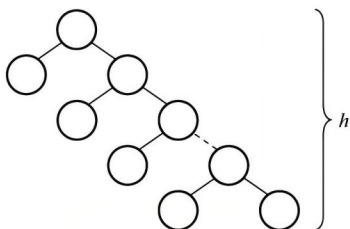
由二叉树的性质 $n_0 = n_2 + 1$ ，得 $n_2 = n_0 - 1 = 10 - 1 = 9$ 。

【另解】画出草图，如下图所示。首先画出 10 个叶结点，然后每 2 个结点向上合并，构造一个新的度为 2 的分支结点，直到构成如下图所示的二叉树，其中度为 2 的分支结点数为 9。



05. B

结点最少的情况如下图所示。除根结点层只有 1 个结点外，其他 $h-1$ 层均有两个结点，结点总数 $= 2(h-1) + 1 = 2h-1$ 。



06. D

除根结点外，在其余 $n-1$ 个结点中，每个结点要么是其父结点的左孩子，要么是其父结点的右孩子，每个结点都有两种可能， $n-1$ 个结点共有 2^{n-1} 种不同的组合形态。

07. C

要求满足条件的树，分析可知当这 50 个结点构成一棵完全二叉树时高度最小， $h = \lfloor \log_2 n \rfloor + 1 = \lfloor \log_2 50 \rfloor + 1 = 6$ 。

【另解】第 1 层最多有 1 个结点，第 2 层最多有 2^1 个结点，第 3 层最多有 2^2 个结点，第 4 层最多有 2^3 个结点，以此类推，可以得到 h 最少为 6。

08. C

由二叉树的性质 1 可知 $n_0 = n_2 + 1$ ，结点总数 $= 2n = n_0 + n_1 + n_2 = n_1 + 2n_2 + 1$ ，则 $n_1 = 2(n - n_2) - 1$ ，所以 n_1 为奇数，说明该二叉树中不可能有 $2m$ 个度为 1 的结点。

09. C

当二叉树为单支树时具有最大高度，即每层上只有一个结点，最大高度为 1025。而当树为完全二叉树时，其高度最小，最小高度为 $\lfloor \log_2 n \rfloor + 1 = 11$ 。

10. C

建议画图，第一层有 1 个结点，其余 $h-1$ 层各有 2 个结点，总结点数 $= 1 + 2(h-1) = 15$ ， $h = 8$ 。

11. C

高度为 h 的完全二叉树中，第 1 层~第 $h-1$ 层构成一个高度为 $h-1$ 的满二叉树，结点数为 $2^{h-1} - 1$ 。第 h 层至少有一个结点，所以最少的结点数 $= (2^{h-1} - 1) + 1 = 2^{h-1}$ 。

```

        InThread(p->lchild,pre);    //递归, 线索化左子树
        if(p->lchild==NULL){        //当前结点的左子树为空
            p->lchild=pre;           //建立当前结点的前驱线索
            p->ltag=1;
        }
        if(pre!=NULL&&pre->rchild==NULL){ //前驱结点非空且其右子树为空
            pre->rchild=p;           //建立前驱结点的后继线索
            pre->rtag=1;
        }
        pre=p;                      //标记当前结点成为刚刚访问过的结点
        InThread(p->rchild,pre);    //递归, 线索化右子树
    }
}

```

建立中序线索二叉树的主过程如下:

```

void CreateInThread(ThreadTree T){
    ThreadTree pre=NULL;
    if(T!=NULL){                //非空二叉树, 线索化
        InThread(T,pre);        //线索化二叉树
        pre->rchild=NULL;       //处理遍历的最后一个结点
        pre->rtag=1;
    }
}

```

为方便遍历, 可在线索链表上增设一个头结点: 其 lchild 指向根结点; rchild 指向中序序列的最后一个结点; 中序序列的第一个结点的 lchild 和最后一个结点的 rchild 均指向头结点。如此, 便形成一个双向循环线索链表, 从而支持正向与反向的高效遍历, 如图 5.18 所示。

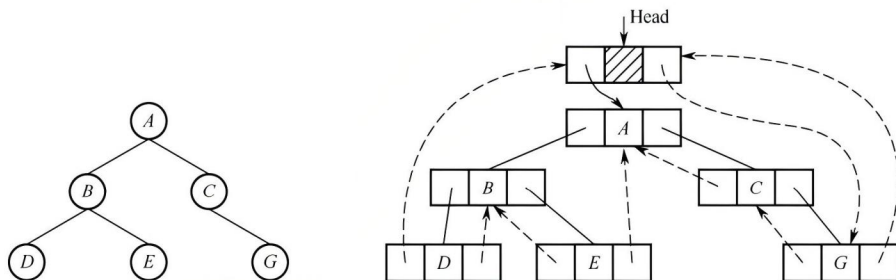


图 5.18 带头结点的中序线索二叉树

3. 中序线索二叉树的遍历

中序线索二叉树的结点中已隐含前驱和后继信息, 因此遍历时无须借助栈或递归。只需先找到中序序列的第一个结点, 然后依次查找每个结点的后继, 直至后继为空 (或回到头结点)。在中序线索二叉树中, 查找结点后继的规则如下: 若结点的 rtag 为 1, 则 rchild 直接指向其后继; 若 rtag 为 0, 则其后继为右子树中最左下结点 (右子树的中序第一个结点)。

不含头结点的中序线索二叉树遍历相关算法如下。

1) 求中序序列中的第一个结点:

```

ThreadNode *Firstnode(ThreadNode *p){
    while(p->ltag==0) p=p->lchild; //沿左孩子链走到最左下结点
    return p;
}

```

2) 求结点 p 在中序序列中的后继:

```

ThreadNode *Nextnode(ThreadNode *p){
    if(p->rtag==0) return Firstnode(p->rchild); //右子树中最左下结点
    else return p->rchild; //若 rtag==1 则直接返回后继线索
}

```

请读者自行完成求中序序列中的最后一个结点和结点 p 的中序前驱的运算^①。

3) 利用上面两个算法, 可实现非递归的中序遍历:

```
void Inorder(ThreadNode *T){
    for(ThreadNode *p=Firstnode(T);p!=NULL; p=Nextnode(p))
        visit(p);
}
```

4. 先序线索二叉树和后序线索二叉树

建立先序或后序线索二叉树的方法与中序类似, 只需调整递归调用顺序及线索建立的位置。以图 5.19(a) 的二叉树为例, 手工构造先序线索二叉树的过程如下(先序序列为 $ABCDF$, 依次处理每个结点, 若其左或右指针为空, 则将其替换为对应前驱或后继的线索):

结点 A 、 B 均有左右孩子, 无须修改指针; 结点 C 无左孩子, 将其 $lchild$ 指向前驱 B , 无右孩子, 将其 $rchild$ 指向后继 D ; 结点 D 无左孩子, 将其 $lchild$ 指向前驱 C , 无右孩子, 将其 $rchild$ 指向后继 F ; 结点 F 无左孩子, 将其 $lchild$ 指向前驱 D , 无右孩子且无后继, 将其 $rchild$ 置为空, 最终得到的先序线索二叉树如图 5.19(b) 所示。

构造后序线索二叉树的过程类似(后序序列为 $CDBFA$):

结点 C 无左孩子且无前驱, 将 $lchild$ 置空, 无右孩子, 将 $rchild$ 指向后继 D ; 结点 D 无左孩子, 将 $lchild$ 指向前驱 C , 无右孩子, 将 $rchild$ 指向后继 B ; 结点 F 无左孩子, 将 $lchild$ 指向前驱 B , 无右孩子, 将 $rchild$ 指向后继 A , 得到的后序线索二叉树如图 5.19(c) 所示。

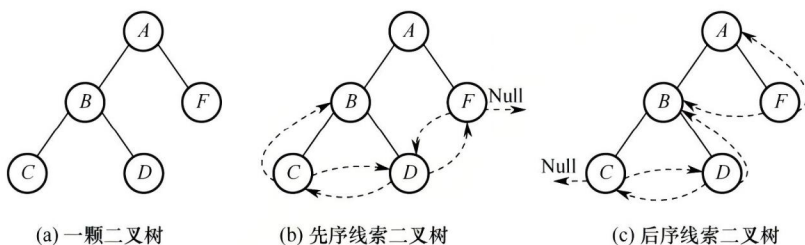


图 5.19 先序线索二叉树和后序线索二叉树

在先序线索二叉树中查找某结点的后继较为直接: 若该结点有左孩子, 则左孩子为其后继; 若无左孩子但有右孩子, 则右孩子为其后继; 若为叶结点, 则其 $rchild$ 直接指向后继。

考点追踪 ▶ 后序线索二叉树的特点 (2013)

而在后序线索二叉树中查找后继则较为复杂, 需要分以下三种情况: ① 若结点 x 是整棵二叉树的根, 则其后继为空; ② 若结点 x 是其双亲的右孩子, 或是其双亲的左孩子且该双亲无右子树, 则 x 的后继即为其双亲; ③ 若结点 x 是其双亲的左孩子, 且该双亲存在右子树, 则 x 的后继为该右子树中按后序遍历访问的第一个结点(右子树中最左下结点)。

值得注意的是, 图 5.19(c) 中, 结点 B 的后继是 A , 但无法直接通过线索找到后继, 需要回溯到其双亲。因此, 在后序线索二叉树中高效查找后继通常需要知道结点的父结点信息。为此, 通常采用带父指针的三叉链表作为存储结构, 而非标准的二叉线索链表。

5.3.3 本节试题精选

一、单项选择题

01. 在下列关于二叉树遍历的说法中, 正确的是 ()。

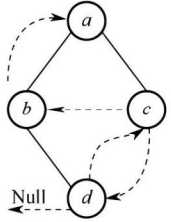
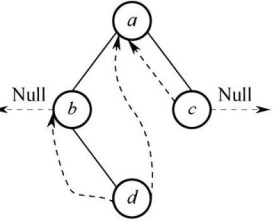
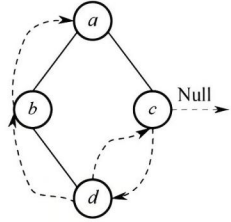
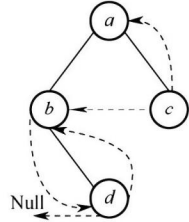
① 将函数 1 中的 $ltag$ 和 $lchild$ 替换为 $rtag$ 和 $rchild$, 并向右下方查找, 即为求中序序列的最后一个结点。将函数 2 中的 $rtag$ 和 $rchild$ 替换为 $ltag$ 和 $lchild$, 调用函数替换为求左子树的中序序列的最后一个结点, 即为求中序序列中结点 p 的前驱。

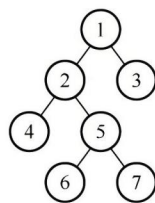
- A. 若有一个结点是二叉树中某个子树的中序遍历结果序列的最后一个结点, 则它一定是该子树的先序遍历结果序列的最后一个结点
- B. 若有一个结点是二叉树中某个子树的先序遍历结果序列的最后一个结点, 则它一定是该子树的中序遍历结果序列的最后一个结点
- C. 若有一个叶结点是二叉树中某个子树的中序遍历结果序列的最后一个结点, 则它一定是该子树的先序遍历结果序列的最后一个结点
- D. 若有一个叶结点是二叉树中某个子树的先序遍历结果序列的最后一个结点, 则它一定是该子树的中序遍历结果序列的最后一个结点
02. 在任何一棵二叉树中, 若结点 a 有左孩子 b 、右孩子 c , 则在结点的先序序列、中序序列、后序序列中, ()。
- A. 结点 b 一定在结点 a 的前面 B. 结点 a 一定在结点 c 的前面
- C. 结点 b 一定在结点 c 的前面 D. 结点 a 一定在结点 b 的前面
03. 设 n, m 为一棵二叉树上的两个结点, 在中序遍历时, n 在 m 前的条件是 ()。
- A. n 在 m 右方 B. n 是 m 祖先 C. n 在 m 左方 D. n 是 m 子孙
04. 设 n, m 为一棵二叉树上的两个结点, 在后序遍历时, n 在 m 前的充分条件是 ()。
- A. n 在 m 右方 B. n 是 m 祖先 C. n 在 m 左方 D. n 是 m 子孙
05. 某非空二叉树采用顺序存储结构, 树中的结点信息按完全二叉树的层次序列依次存放在如下所示的一维数组中, 则该二叉树的后序遍历序列为 ()。

0	1	2	3	4	5	6	7	8	9	10	11	12
a	b	c		d	e	f			g			h

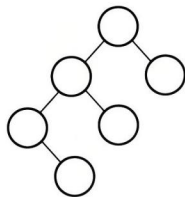
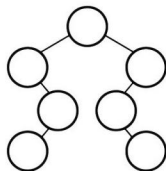
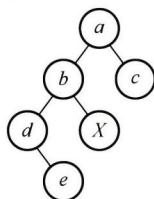
- A. $ghbfehcfa$ B. $gbdehcfa$ C. $gdbhefca$ D. $bgdehcfa$
06. 在二叉树的先序序列、中序序列和后序序列中, 所有叶结点的先后顺序 ()。
- A. 都不相同 B. 完全相同
- C. 先序和中序相同, 而与后序不同 D. 中序和后序相同, 而与先序不同
07. 对二叉树的结点从 1 开始进行连续编号, 要求每个结点的编号大于其左右孩子的编号, 同一结点的左右孩子中, 其左孩子的编号小于其右孩子的编号, 可采用 () 次序的遍历实现编号。
- A. 先序遍历 B. 中序遍历 C. 后序遍历 D. 层次遍历
08. 按某种顺序对二叉树的结点进行编号, 编号为 $1, 2, \dots, n$, 规定: 树中任一结点 v , 其编号等于 v 的左子树上的最小编号减 1, 而 v 的右子树中的最小编号等于 v 的左子树上的最大编号加 1, 则说明该二叉树是按 () 次序编号的。
- A. 中序遍历 B. 先序遍历 C. 后序遍历 D. 层次遍历
09. 先序序列为 A, B, C , 后序序列为 C, B, A 的二叉树共有 ()。
- A. 1 棵 B. 2 棵 C. 3 棵 D. 4 棵
10. 一棵完全二叉树的后序遍历序列为 $CDBFGEA$, 则其先序遍历序列是 ()。
- A. $CBDAFEG$ B. $ABECDFG$ C. $ABCDEF G$ D. 无法确定
11. 设结点 X 和 Y 是二叉树中任意的两个结点。在该二叉树的先序遍历序列中 X 在 Y 之前, 而在其后序遍历序列中 X 在 Y 之后, 则 X 和 Y 的关系是 ()。
- A. X 是 Y 的左兄弟 B. X 是 Y 的右兄弟
- C. X 是 Y 的祖先 D. X 是 Y 的后裔

12. 若二叉树中结点的先序序列是... a ... b ..., 中序序列是... b ... a ..., 则 ()。
- A. 结点 a 和结点 b 分别在某结点的左子树和右子树中
B. 结点 b 在结点 a 的右子树中
C. 结点 b 在结点 a 的左子树中
D. 结点 a 和结点 b 分别在某结点的两棵非空子树中
13. 一棵二叉树的先序遍历序列为 1234567, 它的中序遍历序列可能是 ()。
- A. 3124567 B. 1234567 C. 4135627 D. 1463572
14. 下列序列中, 不能唯一地确定一棵二叉树的是 ()。
- A. 层次序列和中序序列 B. 先序序列和中序序列
C. 后序序列和中序序列 D. 先序序列和后序序列
15. 若一棵二叉树的中序序列和后序序列相同, 则 ()。
- A. 二叉树为空树或二叉树任一结点没有左子树
B. 二叉树为空树或二叉树任一结点没有右子树
C. 二叉树为空树或二叉树中每个结点的度为 1
D. 二叉树为空树或二叉树为满二叉树
16. 已知一棵二叉树的后序序列为 DABEC, 中序序列为 DEBAC, 则先序序列为 ()。
- A. ACBED B. DECAB C. DEABC D. CEDBA
17. 已知一棵二叉树的先序遍历结果为 ABCDEF, 中序遍历结果为 CBAEDF, 则后序遍历的结果为 ()。
- A. CBEFDA B. FEDCBA C. CBEDFA D. 不确定
18. 已知一棵二叉树的层次序列为 ABCDEF, 中序序列为 BADCFE, 则先序序列为 ()。
- A. ACBEDF B. ABCDEF C. BDFECA D. FCEDBA
19. 某二叉树中结点 x 在先序、中序、后序遍历序列中的编号分别为 $\text{pre}(x)$ 、 $\text{in}(x)$ 、 $\text{post}(x)$ (假设都从 1 开始依次顺序编号), a 和 b 是该二叉树中的两个结点, 其中 a 是 b 的祖先, 则下列选项中不可能出现的是 ()。
- A. $\text{pre}(a) < \text{pre}(b)$ B. $\text{post}(a) < \text{post}(b)$ C. $\text{in}(a) < \text{in}(b)$ D. $\text{in}(a) > \text{in}(b)$
20. 某二叉树采用二叉链表存储结构, 若要删除该二叉链表中的所有结点, 并释放它们占用的存储空间, 则采用 () 遍历方法最合适。
- A. 中序 B. 层次 C. 后序 D. 先序
21. 某二叉树 T 采用二叉链表存储结构, T 的中序遍历序列为一个升序序列, 要求采用某种方法对 T 进行某种操作之后得到一棵新的二叉树 T' , 要求 T' 的中序遍历序列为一个降序序列, 则下列关于该算法的叙述中, 正确的是 ()。
- A. 采用中序遍历的方法最合适 B. 采用后序遍历的方法最合适
C. T' 中的根结点一定不是原 T 中的根结点 D. T' 中的叶结点不一定是原 T 中的叶结点
22. 引入线索二叉树的目的是 ()。
- A. 加快查找结点的前驱或后继的速度 B. 为了能在二叉树中方便插入和删除
C. 为了能方便找到双亲 D. 使二叉树的遍历结果唯一
23. n 个结点的线索二叉树上含有的线索数为 ()。
- A. $2n$ B. $n-1$ C. $n+1$ D. n
24. 判断线索二叉树中 $*p$ 结点有右孩子结点的条件是 ()。
- A. $p != \text{NULL}$ B. $p \rightarrow \text{rchild} != \text{NULL}$

- C. $p \rightarrow rtag == 0$ D. $p \rightarrow rtag == 1$
25. 一棵左子树为空的二叉树在先序线索化后, 其中空的链域的个数是 ()。
- A. 不确定 B. 0 个 C. 1 个 D. 2 个
26. 在线索二叉树中, 下列说法不正确的是 ()。
- A. 在中序线索树中, 若某结点有右孩子, 则其后继结点是它的右子树的最左下结点
B. 在中序线索树中, 若某结点有左孩子, 则其前驱结点是它的左子树的最右下结点
C. 线索二叉树是利用二叉树的 $n+1$ 个空指针来存放结点的前驱和后继信息的
D. 每个结点通过线索都可以直接找到它的前驱和后继
27. 二叉树在线索化后, 仍不能有效求解的问题是 ()。
- A. 先序线索二叉树中求先序后继 B. 中序线索二叉树中求中序后继
C. 中序线索二叉树中求中序前驱 D. 后序线索二叉树中求后序后继
28. 若 X 是二叉中序线索树中一个有左孩子的结点, 且 X 不为根, 则 X 的前驱为 ()。
- A. X 的双亲 B. X 的右子树中最左的结点
C. X 的左子树中最右的结点 D. X 的左子树中最右的叶结点
29. 若 X 是后序线索二叉树中的叶结点, 且 X 存在左兄弟 Y , 则 X 的右线索指向的是 ()。
- A. X 的双亲 B. 以 Y 为根的子树的最左下结点
C. X 的左兄弟 Y D. 以 Y 为根的子树的最右下结点
30. 某二叉树的先序序列和后序序列正好相反, 则该二叉树一定是 ()。
- A. 空或只有一个结点 B. 高度等于其结点数
C. 任意一个结点无左孩子 D. 任意一个结点无右孩子
31. 某非空二叉树的先序序列和中序序列正好相反, 则下列叙述中正确的是 ()。
- A. 该二叉树一定只有一个结点
B. 只有一个叶结点的二叉树一定满足条件
C. 任意一个结点无左孩子的二叉树一定满足条件
D. 任意一个结点无右孩子的二叉树一定满足条件
32. 【2009 统考真题】给定二叉树如右图所示。设 N 代表二叉树的根, L 代表根结点的左子树, R 代表根结点的右子树。若遍历后的结点序列是 3175624, 则其遍历方式是 ()。
- A. LRN B. NRL C. RLN D. RNL
33. 【2010 统考真题】下列线索二叉树中 (用虚线表示线索), 符合后序线索树定义的是 ()。
- A.  B.  C.  D. 
34. 【2011 统考真题】一棵二叉树的先序遍历序列和后序遍历序列分别为 1, 2, 3, 4 和 4, 3, 2, 1, 该二叉树的中序遍历序列不会是 ()。
- A. 1, 2, 3, 4 B. 2, 3, 4, 1 C. 3, 2, 4, 1 D. 4, 3, 2, 1



35. 【2012 统考真题】若一棵二叉树的先序遍历序列为 a, e, b, d, c , 后序遍历序列为 b, c, d, e, a , 则根结点的孩子结点 ()。
- A. 只有 e B. 有 e, b C. 有 e, c D. 无法确定
36. 【2013 统考真题】若 X 是后序线索二叉树中的叶结点, 且 X 存在左兄弟结点 Y , 则 X 的右线索指向的是 ()。
- A. X 的父结点 B. 以 Y 为根的子树的最左下结点
C. X 的左兄弟结点 Y D. 以 Y 为根的子树的最右下结点
37. 【2014 统考真题】若对右图所示的二叉树进行中序线索化, 则结点 X 的左右线索指向的结点分别是 ()。
- A. e, c B. e, a C. d, c D. b, a
38. 【2015 统考真题】先序序列为 a, b, c, d 的不同二叉树的个数是 ()。
- A. 13 B. 14 C. 15 D. 16
39. 【2017 统考真题】某二叉树的树形如右图所示, 其后序序列为 e, a, c, b, d, g, f , 树中与结点 a 同层的结点是 ()。
- A. c B. d C. f D. g
40. 【2017 统考真题】要使一棵非空二叉树的先序序列与中序序列相同, 其所有非叶结点须满足的条件是 ()。
- A. 只有左子树 B. 只有右子树
C. 结点的度均为 1 D. 结点的度均为 2
41. 【2022 统考真题】若结点 p 与 q 在二叉树 T 的中序遍历序列中相邻, 且 p 在 q 之前, 则下列 p 与 q 的关系中, 不可能的是 ()。
- I. q 是 p 的双亲 II. q 是 p 的右孩子
III. q 是 p 的右兄弟 IV. q 是 p 的双亲的双亲
A. 仅 I B. 仅 III C. 仅 II、III D. 仅 II、IV
42. 【2023 统考真题】已知一棵二叉树的树形如右图所示, 若其后序遍历序列为 $fdbeca$, 则其先 (前) 序遍历序列是 ()。
- A. $aedfbc$ B. $acebdf$
C. $cabefd$ D. $dfebac$
43. 【2024 统考真题】若 p, q 和 v 均为二叉树 T 中的结点, v 有两个孩子结点, T 的中序遍历序列形如 “ $\cdots, p, v, q, \cdots$ ”, 则在下列叙述中, 正确的是 ()。
- A. p 没有右孩子, q 没有左孩子 B. p 没有右孩子, q 有左孩子
C. p 有右孩子, q 没有左孩子 D. p 有右孩子, q 有左孩子



二、综合应用题

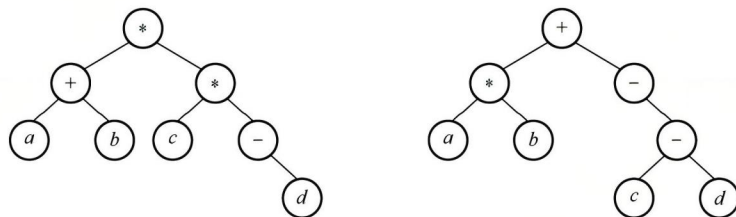
- 若某非空二叉树的先序序列和后序序列正好相反, 则该二叉树的形态是什么?
- 若某非空二叉树的先序序列和后序序列正好相同, 则该二叉树的形态是什么?
- 假设二叉树采用二叉链表存储结构, 设计一个非递归算法求二叉树的高度。
- 二叉树按二叉链表形式存储, 试编写一个判别给定二叉树是否是完全二叉树的算法。
- 假设二叉树采用二叉链表存储结构存储, 试设计一个算法, 计算一棵给定二叉树的所有双分支结点数。
- 设树 B 是一棵采用链式结构存储的二叉树, 编写一个把树 B 中所有结点的左右子树进行交换的函数。

07. 假设二叉树采用二叉链存储结构存储, 设计一个算法, 求先序遍历序列中第 k ($1 \leq k \leq$ 二叉树中结点数) 个结点的值。
08. 已知二叉树以二叉链表存储, 编写算法完成: 对于树中每个元素值为 x 的结点, 删除以它为根的子树, 并释放相应的空间。
09. 在二叉树中查找值为 x 的结点, 试编写算法 (用 C 语言) 打印值为 x 的结点的所有祖先, 假设值为 x 的结点不多于一个。
10. 设一棵二叉树的结点结构为 (LLINK, INFO, RLINK), ROOT 为指向该二叉树根结点的指针, p 和 q 分别为指向该二叉树中任意两个结点的指针, 试编写算法 ANCESTOR (ROOT, p , q , r), 找到 p 和 q 的最近公共祖先结点 r 。
11. 假设二叉树采用二叉链表存储结构, 设计一个算法, 求非空二叉树 b 的宽度 (具有结点数最多的那一层的结点数)。
12. 设有一棵满二叉树 (所有结点值均不同), 已知其先序序列为 pre , 设计一个算法求其后序序列 $post$ 。
13. 设计一个算法将二叉树的叶结点按从左到右的顺序连成一个单链表, 表头指针为 $head$ 。二叉树按二叉链表方式存储, 链接时用叶结点的右指针域来存放单链表指针。
14. 试设计判断两棵二叉树是否相似的算法。所谓二叉树 T_1 和 T_2 相似, 指的是 T_1 和 T_2 都是空的二叉树或都只有一个根结点; 或者 T_1 的左子树和 T_2 的左子树是相似的, 且 T_1 的右子树和 T_2 的右子树是相似的。
15. 【2014 统考真题】二叉树的带权路径长度 (WPL) 是二叉树中所有叶结点的带权路径长度之和。给定一棵二叉树 T , 采用二叉链表存储, 结点结构为

left	weight	right
------	--------	-------

其中叶结点的 $weight$ 域保存该结点的非负权值。设 $root$ 为指向 T 的根结点的指针, 请设计求 T 的 WPL 的算法, 要求:

- 1) 给出算法的基本设计思想。
 - 2) 使用 C 或 C++ 语言, 给出二叉树结点的数据类型定义。
 - 3) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
16. 【2017 统考真题】请设计一个算法, 将给定的表达式树 (二叉树) 转换为等价的中缀表达式 (通过括号反映操作符的计算次序) 并输出。例如, 当下列两棵表达式树作为算法的输入时:



输出的等价中缀表达式分别为 $(a+b) * (c * (-d))$ 和 $(a*b) + (- (c-d))$ 。

二叉树结点定义如下:

```
typedef struct node{
    char data[10];           //存储操作数或操作符
    struct node *left, *right;
} BTree;
```

要求:

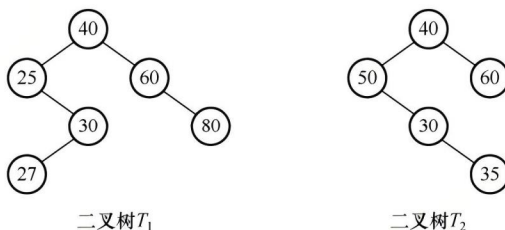
- 1) 给出算法的基本设计思想。

2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。

17. 【2022 统考真题】已知非空二叉树 T 的结点值均为正整数, 采用顺序存储方式保存, 数据结构定义如下:

```
typedef struct {
    int SqBiTNode[MAX_SIZE]; //MAX_SIZE 为已定义常量
    int ElemNum;              //保存二叉树结点值的数组
                              //实际占用的数组元素个数
} SqBiTree;
```

T 中不存在的结点在数组 SqBiTNode 中用 -1 表示。例如, 对于下图所示的两棵非空二叉树 T_1 和 T_2 ,



T_1 的存储结果如下:

T1.SqBiTNode	40	25	60	-1	30	-1	80	-1	-1	27		
T1.ElemNum=10												

T_2 的存储结果如下:

T2.SqBiTNode	40	50	60	-1	30	-1	-1	-1	-1	-1	35	
T2.ElemNum=11												

请设计一个尽可能高效的算法, 判定一棵采用这种方式存储的二叉树是否为二叉搜索树, 若是, 则返回 true, 否则, 返回 false。要求:

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。

5.3.4 答案与解析

一、单项选择题

01. C

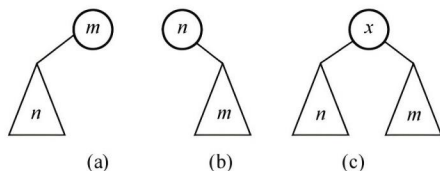
二叉树中序遍历的最后一个结点一定是从根开始沿右孩子指针链走到底的结点, 设用 p 指示。若结点 p 不是叶结点 (其左子树非空), 则先序遍历的最后一个结点在它的左子树中, 选项 A、B 错误; 若结点 p 是叶结点, 则先序与中序遍历的最后一个结点就是它, 选项 C 正确。若中序遍历的最后一个结点 p 不是叶结点, 它还有一个左孩子 q , 结点 q 是叶结点, 那么结点 q 是先序遍历的最后一个结点, 但不是中序遍历的最后一个结点, 选项 D 错误。

02. C

三种遍历方式中, 都先遍历左子树, 再遍历右子树, 因此 b 一定在 c 的前面访问。

03. C

中序遍历时, 先访问左子树, 再访问根结点, 后访问右子树。 n 在 m 前的 3 种可能性如右图所示, 从中看出 n 总是在 m 的左方。



【另解】设 n 和 m 的最近公共祖先 p , 则有以下可能:

情形 1, m 和 n 分别在 p 的左右 (右左) 分支上; 情形 2, m 或 n 为 p 结点, 另一结点在 p

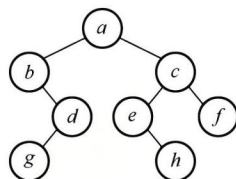
的分支上。只有 n 和 m 分别处于 p 的左右分支上, m 为祖先结点且 n 位于 m 的左分支, n 为祖先结点且 m 位于 n 的右分支, 符合题意。

04. D

后序遍历的顺序是 LRN, 若 n 在 N 的左子树上, m 在 N 的右子树上, 则在后序遍历的过程中 n 在 m 之前访问; 若 n 是 m 的子孙, 设 m 在 N 的位置, 则 n 无论是在 m 的左子树还是在右子树, 在后序遍历的过程中 n 都在 m 之前访问。其他都不可以。选项 C 要成立, 就需加上两个结点位于同一层这个条件。

05. C

在二叉树的数组存储结构中, 下标为 i 的结点的左右孩子的下标分别为 $2i+1$ 和 $2i+2$ (若存在), 画出二叉树的形态如下右图所示, 则后序遍历序列为 $gdbhefca$ 。



06. B

三种遍历方式中, 访问左右子树的先后顺序是不变的, 只是访问根结点的顺序不同, 因此叶结点的先后顺序完全相同。此外, 读者可以采用特殊值法, 画一个结点数为 3 的满二叉树, 采用三种遍历方式来验证答案的正确性。

07. C

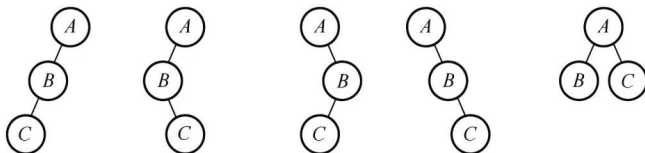
对每个顶点从 1 开始按序编号, 要求结点编号大于其左右孩子编号, 并且左孩子编号小于右孩子编号。编号越大说明遍历顺序越靠后, 因此, 三者遍历顺序为先左子树、再右子树、后根结点。4 个选项中仅后序遍历满足要求。

08. B

结点 v 的编号比其左子树上的最小编号还小, 而 v 的右子树中的最小编号大于 v 的左子树中的最大编号, 因此 v 的编号比其左右子树上的所有编号都小, 显然是按先序遍历次序。

09. D

先序为 A 、 B 、 C 的不同二叉树共有 5 种, 其中后序为 C 、 B 、 A 的有 4 种 (前 4 种), 都是单支树, 如下图所示。



10. C

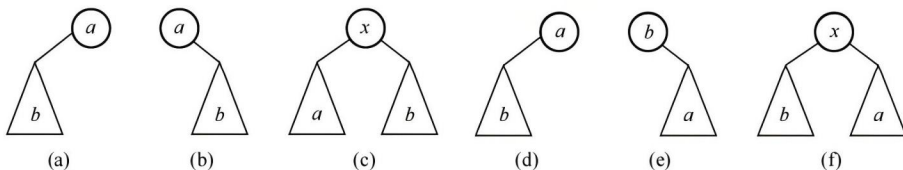
7 个结点的完全二叉树是一棵 3 层的满二叉树, 画出相应二叉树的树形, 根据后序遍历序列填入相应的结点, 得到相应的完全二叉树, 求得其先序遍历序列为 $ABCDEFG$ 。

11. C

二叉树的先序遍历为 NLR, 后序遍历为 LRN。根据题意, 在先序序列中 X 在 Y 之前, 在后序序列中 X 在 Y 之后, 若设 X 在根的位置, Y 在其左子树或右子树中, 即满足要求。

12. C

先序序列是 $\dots a \dots b \dots$, 因此 a 和 b 结点的 3 种情况如下图(a)~(c)所示。中序序列是 $\dots b \dots a \dots$, 因此 a 和 b 结点的 3 种情况如下图(d)~(f)所示, 相同部分是 b 在 a 的左子树中。



对应的二叉树；由于每棵转换后的二叉树根结点均无右子树，可将这些根结点视为兄弟；将第二棵二叉树作为第一棵二叉树根的右子树……以此类推，最终形成一棵完整的二叉树。

森林转换为二叉树的手工画法：

- 1) 将森林中的每棵树转换为相应的二叉树；
- 2) 每棵树的根也可视为兄弟关系，在各树的根之间依次添加连线；
- 3) 以第一棵树的根为轴心，顺时针旋转 45° 。

等效做法是：先在各树的根之间添加连线，然后整体应用树转二叉树的方法处理。

3. 二叉树转换为森林

考点追踪 ▶ 由遍历序列构造一棵二叉树并转换为对应的森林（2020、2021）

若给定的二叉树非空，则其转换为森林的规则：二叉树的根及其左子树对应第一棵树的二叉树表示；将根的右指针断开，其右子树即代表剩余森林转换后的二叉树；对右子树递归应用相同规则，不断分离出下一棵树，直至右子树为空；最后将每棵分离出的二叉树还原为普通树，即可得到原始森林。此过程如图 5.23 所示。二叉树转换为树或森林的结果是唯一的。

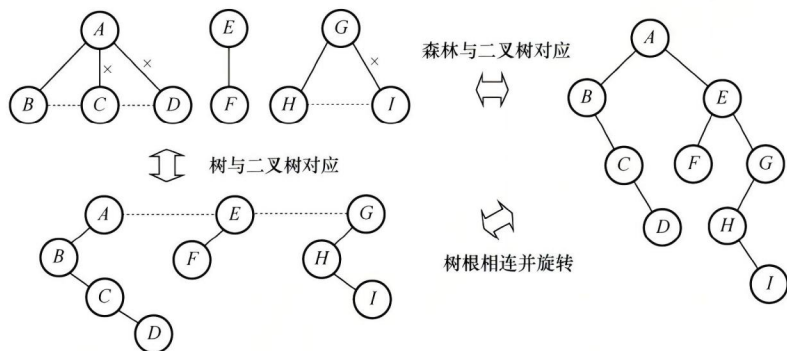


图 5.23 森林与二叉树的对应关系

5.4.3 树和森林的遍历

1. 树的遍历

考点追踪 ▶ 树与二叉树遍历方法的对应关系（2019）

树的遍历是指以某种方式访问树中的每个结点，且仅访问一次。主要有两种方式：

- 1) 先根遍历。若树非空，则按如下规则遍历：

- 先访问根结点。
 - 再依次遍历根结点的每棵子树，遍历子树时仍遵循先根后子树的规则。
- 其遍历序列与这棵树相应二叉树的先序序列相同。

- 2) 后根遍历。若树非空，则按如下规则遍历：

- 先依次遍历根结点的每棵子树，遍历子树时仍遵循先子树后根的规则。
- 再访问根结点。

其遍历序列与这棵树相应二叉树的中序序列相同。

图 5.22 的树的先根遍历序列为 $ABEFCDBG$ ，后根遍历序列为 $EFBCGDA$ 。

此外，树也支持层次遍历，与二叉树的层次遍历原理相似，即按层序依次访问各个结点。

2. 森林的遍历

基于森林和树相互递归的定义，可得出森林的两种遍历方法。

1) 先序遍历森林。若森林为非空, 则按如下规则遍历:

- 访问森林中第一棵树的根结点。
- 先序遍历第一棵树中根结点的子树森林。
- 先序遍历除去第一棵树后的剩余森林。

2) 中序遍历森林。若森林为非空, 则按如下规则遍历:

- 中序遍历第一棵树中根结点的子树森林。
- 访问森林中第一棵树的根结点。
- 中序遍历除去第一棵树后的剩余森林。

图 5.23 的森林的先序遍历序列为 $ABCDEFGH I$, 中序遍历序列为 $BCDAFEHIG$ 。

考点追踪 ▶ 森林与二叉树遍历方法的对应关系 (2020)

当森林转换成二叉树时, 第一棵树的子树森林会转换为左子树, 剩余森林则转换为右子树。因此, 森林的先序、中序遍历与其对应二叉树的先序、中序遍历相同。

树和森林的遍历与二叉树的遍历关系见表 5.1。

表 5.1 树和森林的遍历与二叉树遍历的对应关系

树	森 林	二 叉 树
先根遍历	先序遍历	先序遍历
后根遍历	中序遍历	中序遍历

注 意

森林遍历方法的命名按照严蔚敏老师的教材。有些教材也将森林的中序遍历称为后序遍历, 称中序遍历是相对其二叉树而言的, 称后序遍历是因为根确实是在最后才被访问的。

5.4.4 本节试题精选

一、单项选择题

01. 下列关于树的说法中, 正确的是 ()。

- I. 对于有 n 个结点的二叉树, 其高度为 $\log_2 n$
- II. 完全二叉树中, 若一个结点没有左孩子, 则它必是叶结点
- III. 高度为 h ($h > 0$) 的完全二叉树对应的森林所含的树的个数一定是 h
- IV. 一棵树中的叶子数一定等于与其对应的二叉树的叶子数

A. I 和 III B. IV C. I 和 II D. II

02. 利用二叉链表存储森林时, 根结点的右指针是 ()。

A. 指向最左兄弟 B. 指向最右兄弟 C. 一定为空 D. 不一定为空

03. 设森林 F 中有 3 棵树, 第 1、2、3 棵树的结点数分别为 M_1 , M_2 和 M_3 , 与森林 F 对应的二叉树根结点的右子树上的结点数是 ()。

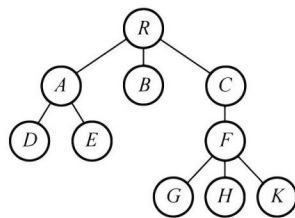
A. M_1 B. $M_1 + M_2$ C. M_3 D. $M_2 + M_3$

04. 设森林 F 中有 4 棵树, 第 1、2、3、4 棵树的结点数分别为 a 、 b 、 c 和 d , 与森林 F 对应的二叉树的根结点的左子树上的结点数是 ()。

A. a B. $b + c + d$ C. $a - 1$ D. $a + b + c$

05. 设森林 F 对应的二叉树为 B , 它有 m 个结点, B 的根为 p , p 的右子树结点数为 n , 森林 F 中第一棵树的结点数是 ()。

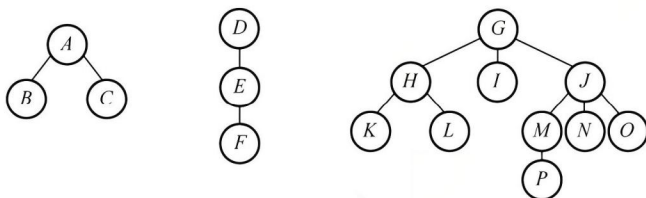
- A. $m-n$ B. $m-n-1$ C. $n+1$ D. 条件不足, 无法确定
06. 设森林 F 对应的二叉树是一棵具有 16 个结点的完全二叉树, 则森林 F 中树的数目和结点最多的树的结点数分别是 ()。
- A. 2 和 8 B. 2 和 9 C. 4 和 8 D. 4 和 9
07. 森林 $T=(T_1, T_2, \dots, T_m)$ 转化为二叉树 BT 的过程为: 若 $m=0$, 则 BT 为空, 若 $m \neq 0$, 则 ()。
- A. 将中间子树 T_{mid} ($\text{mid}=(1+m)/2$) 的根作为 BT 的根; 将 $(T_1, T_2, \dots, T_{\text{mid}-1})$ 转换为 BT 的左子树; 将 $(T_{\text{mid}+1}, \dots, T_m)$ 转换为 BT 的右子树
- B. 将子树 T_1 的根作为 BT 的根; 将 T_1 的子树森林转换成 BT 的左子树; 将 $(T_2, T_3, \dots, T_m)$ 转换成 BT 的右子树
- C. 将子树 T_1 的根作为 BT 的根; 将 T_1 的左子树森林转换成 BT 的左子树; 将 T_1 的右子树森林转换为 BT 的右子树; 其他以此类推
- D. 将森林 T 的根作为 BT 的根; 将 $(T_1, T_2, \dots, T_m)$ 转化为该根下的结点, 得到一棵树, 然后将这棵树再转化为二叉树 BT
08. 设 F 是一个森林, B 是由 F 变换来的二叉树。若 F 中有 n 个非终端结点, 则 B 中右指针域为空的结点有 () 个。
- A. $n-1$ B. n C. $n+1$ D. $n+2$
09. 设某树的孩子兄弟链表示中共有 6 个空的左指针域、7 个空的右指针域, 包括 5 个结点的左右指针域都为空, 则该树中叶结点的个数是 ()。
- A. 7 B. 6 C. 5 D. 不能确定
10. 若 T_1 是由有序树 T 转换而来的二叉树, 则 T 中结点的后根序列就是 T_1 中结点的 () 序列。
- A. 先序 B. 中序 C. 后序 D. 层序
11. 某二叉树结点的中序序列为 $BDAECF$, 后序序列为 $DBEFCA$, 则该二叉树对应的森林包括 () 棵树。
- A. 1 B. 2 C. 3 D. 4
12. 设 X 是树 T 中的一个非根结点, B 是 T 所对应的二叉树。在 B 中, X 是其双亲结点的右孩子, 下列结论中正确的是 ()。
- A. 在树 T 中, X 是其双亲结点的第一个孩子
- B. 在树 T 中, X 一定无右边兄弟
- C. 在树 T 中, X 一定是叶结点
- D. 在树 T 中, X 一定有左边兄弟
13. 右图是一棵逻辑上的树 T , 则在关于该树的存储结构的叙述中, 错误的是 ()。
- A. 若 T 采用双亲表示法, 则有 9 个指向双亲的指针
- B. 若 T 采用孩子表示法, 则在 T 中查找某个结点的孩子比双亲表示法更方便
- C. 若 T 采用孩子兄弟表示法, 则在 T 中查找某个结点的双亲的时间复杂度为 $O(1)$
- D. 双亲表示法是顺序存储结构, 孩子表示法和孩子兄弟表示法通常是链式存储结构
14. 在森林的二叉树表示中, 结点 M 和结点 N 是同一父结点的左孩子和右孩子, 则在森林中 ()。



- A. M 和 N 有同一双亲 B. M 和 N 可能无公共祖先
C. M 是 N 的孩子 D. M 是 N 的左兄弟
15. 【2009 统考真题】将森林转换为对应的二叉树, 若在二叉树中, 结点 u 是结点 v 的父结点的父结点, 则在原来的森林中, u 和 v 可能具有的关系是 ()。
I. 父子关系 II. 兄弟关系 III. u 的父结点与 v 的父结点是兄弟关系
A. 只有 II B. I 和 II C. I 和 III D. I、II 和 III
16. 【2011 统考真题】已知一棵有 2011 个结点的树, 其叶结点数为 116, 该树对应的二叉树中无右孩子的结点数是 ()。
A. 115 B. 116 C. 1895 D. 1896
17. 【2014 统考真题】将森林 F 转换为对应的二叉树 T , F 中叶结点的个数等于 ()。
A. T 中叶结点的个数 B. T 中度为 1 的结点数
C. T 中左孩子指针为空的结点数 D. T 中右孩子指针为空的结点数
18. 【2019 统考真题】若将一棵树 T 转化为对应的二叉树 BT, 则下列对 BT 的遍历中, 其遍历序列与 T 的后根遍历序列相同的是 ()。
A. 先序遍历 B. 中序遍历
C. 后序遍历 D. 按层遍历
19. 【2020 统考真题】已知森林 F 及与之对应的二叉树 T , 若 F 的先根遍历序列是 a, b, c, d, e, f , 后根遍历序列是 b, a, d, f, e, c , 则 T 的后序遍历序列是 ()。
A. b, a, d, f, e, c B. b, d, f, e, c, a
C. b, f, e, d, c, a D. f, e, d, c, b, a
20. 【2021 统考真题】某森林 F 对应的二叉树为 T , 若 T 的先序遍历序列是 a, b, d, c, e, g, f , 中序遍历序列是 b, d, a, e, g, c, f , 则 F 中树的棵数是 ()。
A. 1 B. 2 C. 3 D. 4
21. 【2025 统考真题】下列关于二叉树及森林的叙述中, 正确的是 ()。
A. 完全二叉树中不存在度为 1 的结点
B. 任意一个森林都可以转换为一棵二叉树
C. 二叉树的分支结点数比叶结点数少
D. 表达式树的根中保存的是最先计算的运算符

二、综合应用题

01. 给定一棵树的先根遍历序列和后根遍历序列, 能否唯一确定一棵树? 若能, 请举例说明; 若不能, 请给出反例。
02. 将下面一个由 3 棵树组成的森林转换为二叉树。



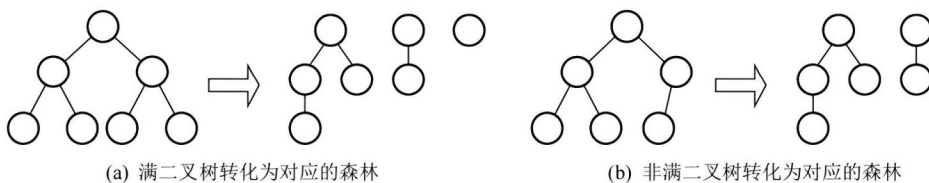
03. 已知某二叉树的先序序列和中序序列分别为 $ABDEHCFIMGJKL$ 和 $DBHEAIMFCGKLJ$, 请画出这棵二叉树, 并画出二叉树对应的森林。
04. 编程求以孩子兄弟表示法存储的森林的叶结点数。
05. 以孩子兄弟链表为存储结构, 请设计递归算法求树的深度。

5.4.5 答案与解析

一、单项选择题

01. D

若 n 个结点的二叉树是一棵单支树, 则其高度为 n 。完全二叉树中最多存在一个度为 1 的结点且只有左孩子, 若不存在左孩子, 则一定也不存在右孩子, 因此必是叶结点, 说法 II 正确。只有满二叉树才具有性质 III, 如下图所示。



在树转换为二叉树时, 若有几个叶结点具有共同的双亲, 则转换成二叉树后只有一个叶结点 (最右边的叶结点), 如下图所示, 说法 IV 错误。注意, 若树中的任意两个叶结点都不存在相同的双亲, 则树中的叶子数才有可能与其对应的二叉树中的叶子数相等。



02. D

森林与二叉树具有对应关系, 因此, 我们存储森林时应先将森林转换成二叉树, 转换的方法就是“左孩子右兄弟”, 与树不同的是, 若存在第二棵树, 则二叉链表的根结点的右指针指向的是森林中的第二棵树的根结点。若此森林只有一棵树, 则根结点的右指针为空。因此, 右指针可能为空也可能不为空。

03. D

与树转换为二叉树不同, 森林中的每棵树是独立的, 因此先要将每棵树的根结点全部视为兄弟结点的关系。森林转换为二叉树后, 树 2 作为树 1 的根结点的右子树, 树 3 作为树 2 的根结点的右子树, 因此森林 F 对应的二叉树根结点的右子树上的结点数是 $M_2 + M_3$ 。

04. C

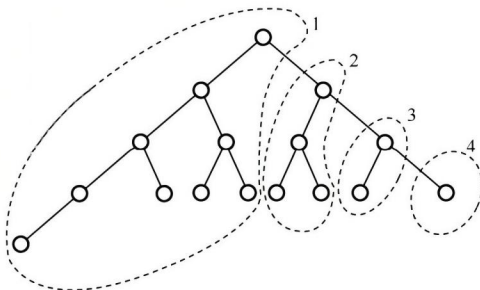
森林转换为二叉树后, 二叉树的根结点为第 1 棵树的根结点, 二叉树的根结点的左子树包含第 1 棵树的所有孩子, 因此森林 F 对应的二叉树的根结点的左子树上的结点数是 $a - 1$ 。

05. A

森林转换成二叉树时采用孩子兄弟表示法, 根结点及其左子树为森林中的第一棵树。右子树为其他剩余的树。所以, 第一棵树的结点数为 $m - n$ 。

06. D

森林转换为二叉树后, 二叉树的根结点及其左子树由第 1 棵树转换得到, 二叉树的根结点的右子树由剩余的森林转换得到, 以此类推, 可以划分出第 2, 3, ... 棵树的结点。具有 16 个结点的完全二叉树的形态如右图所示, 沿二叉树的根结点往右下遍历, 共有 4 个结点, 可知森林中有 4 棵树, 其中第 1 棵树的结点数最多,



有 9 个。

07. B

将森林中每棵树的根结点视为兄弟结点的关系, 再按照“左孩子右兄弟”的规则来进行转化。

08. C

根据森林与二叉树转换规则“左孩子右兄弟”。二叉树 B 中右指针域为空代表该结点没有兄弟结点。森林中每棵树的根结点从第二个开始依次连接到前一棵树的根的右孩子, 因此最后一棵树的根结点的右指针为空。另外, 每个非终端结点, 其所有孩子结点在转换之后, 最后一个孩子的右指针也为空, 所以树 B 中右指针域为空的结点有 $n+1$ 个。

09. B

在树的孩子兄弟表示法中, 若一个结点没有孩子(叶结点), 则表现为该结点的左指针域为空, 因此本题答案为“6”。至于“5 个结点的左右指针域都为空”, 表示树中有 5 个结点既没有孩子又没有右兄弟, 约束条件比题中的“求叶结点的个数”要求更严格。

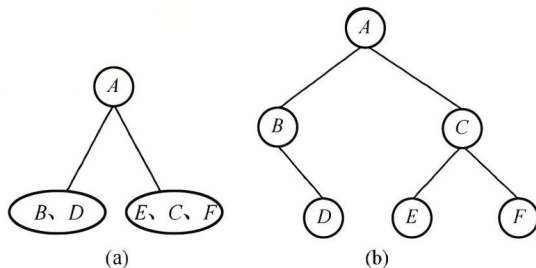
10. B

有序树 T 转换成二叉树 T_1 时, T 的后根序列是对应 T_1 的中序序列还是后序序列呢(显然树的后根序列不可能对应二叉树的先序序列和层序序列)? 看右图所示的例子, 在树 T 中, 叶结点 B 应最先访问, 在 T_1 中, B 的右兄弟 C 转换为它的右孩子, 若对应 T_1 的后序序列, 则 C 应在 B 的前面访问, 所以 T 的后根序列不可能对应 T_1 的后序序列。



11. C

根据二叉树的先序序列和中序序列可以确定一棵二叉树。根据后序序列, A 是二叉树的根结点。根据中序序列, 二叉树的形态如下图(a)所示。对于 A 的左子树, 根据后序序列, B 比 D 后被访问, 因此 B 必为 D 的父结点, 又根据中序序列, D 是 B 的右孩子。对于 A 的右子树, 同理可确定结点 E 、 C 、 F 的关系。此二叉树的形态如下图(b)所示。



再根据二叉树与森林的对应关系, 森林中树的棵数即其对应二叉树(向右旋转 45° 后)的根结点 A 及其右兄弟数, 或解释为: 对应二叉树从根结点 A 开始不断往右孩子访问, 所访问到的结点数。可知此森林中有 3 棵树, 根结点分别为 A 、 C 和 F 。

12. D

在二叉树 B 中, X 是其双亲的右孩子, 因此在树 T 中, X 必是其双亲结点的右兄弟, 换句话说, X 在树中必有左兄弟。

13. C

若 T 采用双亲表示法存储, 则除根结点外, 其余每个结点都有指向其双亲的指针, T 共有 10 个结点, 于是有 9 个指向双亲的指针, 选项 A 正确。若 T 采用孩子表示法存储, 则每个结点的孩子被视为一个线性表, 且以单链表作为存储结构, 只要遍历该单链表, 就能找到某个结点的所有孩子, 而双亲表示法要寻找某个结点的孩子, 就必须遍历整棵树, 选项 B 正确。若 T 采用孩子兄

下面是并查集主要运算的实现。

(1) 并查集的初始化操作

```
void Initial(int S[]){           //s 即并查集
    for(int i=0;i<SIZE;i++){    //每个自成一个单元素集合
        S[i]=-1;
    }
}
```

将每个元素对应的数组值设为-1，表示这些元素各自为根。

(2) 并查集的 Find 操作

在并查集中查找并返回包含元素 x 的树的根。

```
int Find(int S[],int x){
    while(S[x]>=0)                //循环寻找 x 的根
        x=S[x];
    return x;                    //根的 s[] 小于 0
}
```

判断两个元素是否属于同一集合，只需分别找到它们的根，再比较根是否相同即可。

(3) 并查集的 Union 操作

合并两个不相交子集合的操作。当要合并两个元素所在的集合时，需要先找到这两个元素的根，然后让一棵子集树的根指向另一棵子集树的根。

```
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return;      //要求 Root1 与 Root2 是不同的集合
    S[Root2]=Root1;              //将根 Root2 连接到另一根 Root1 下面
}
```

Find 和 Union 操作的时间复杂度分别为 $O(d)$ 和 $O(1)$ ，其中 d 为树的深度。

4. 并查集实现的优化

在极端情况下，由 n 个元素构成的集合树可能退化为一条链，深度达到 n ，此时 Find 操作的最坏时间复杂度为 $O(n)$ 。改进的办法是：在执行 Union 操作前先比较两个子集合的规模，将规模较小的树作为子树挂到规模较大的树的根下，即“小树并入大树”。为此，可利用根结点的绝对值（其绝对值即为集合大小）来记录该集合中包含的元素个数。

(1) 改进的 Union 操作

```
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return;
    if(S[Root2]>S[Root1]){        //Root2 结点数更少
        S[Root1]+=S[Root2];      //累加集合中的元素总数
        S[Root2]=Root1;          //小树合并到大树
    }
    else{                          //Root1 结点数更少
        S[Root2]+=S[Root1];      //累加结点总数
        S[Root1]=Root2;          //小树合并到大树
    }
}
```

采用这种按集合大小合并的策略后，所构造的集合树深度不超过 $\lfloor \log_2 n \rfloor + 1$ 。

然而，随着子集不断合并，集合树的深度仍可能逐渐增大。为进一步缩短查找时间，还可对上述 Find 操作进一步优化：引入路径压缩机制——在找到根结点后，将从 x 到根路径上的所有中间结点直接挂接到根结点下，从而“压平”整条路径。

(2) 改进的 Find 操作

```
int Find(int S[],int x){
    int root=x;
```

```

while(S[root]>=0)           //循环找到根
    root=S[root];
while(x!=root){             //压缩路径
    int t=S[x];              //t 指向 x 的父结点
    S[x]=root;               //x 直接挂到根结点下面
    x=t;
}
return root;                 //返回根结点编号
}

```

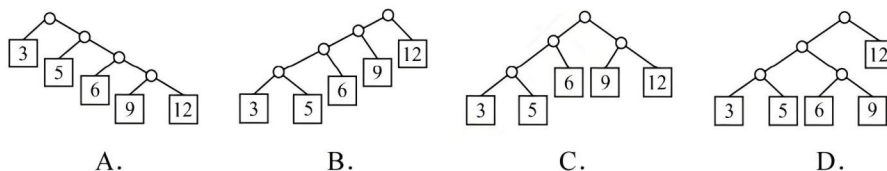
采用按大小合并与路径压缩两种优化后，并查集的操作效率得到极大提升，使得集合树的深度不超过 $O(\alpha(n))$ ，其中 $\alpha(n)$ 是一个增长极其缓慢的函数，对于常见的正整数 n ，通常 $\alpha(n) \leq 4$ 。

关于并查集的实际应用示例，请参见本书配套视频。

5.5.3 本节试题精选

一、单项选择题

01. 在有 n 个叶结点的哈夫曼树中，非叶结点的总数是 ()。
- A. $n-1$ B. n C. $2n-1$ D. $2n$
02. 给定整数集合 $\{3, 5, 6, 9, 12\}$ ，与之对应的哈夫曼树是 ()。



03. 下列编码中，() 不是前缀码。
- A. $\{00, 01, 10, 11\}$ B. $\{0, 1, 00, 11\}$
 C. $\{0, 10, 110, 111\}$ D. $\{10, 110, 1110, 1111\}$
04. 设哈夫曼编码的长度不超过 4，若已对两个字符编码为 1 和 01，则还最多可对 () 个字符编码。
- A. 2 B. 3 C. 4 D. 5
05. 一棵哈夫曼树共有 215 个结点，对其进行哈夫曼编码，共能得到 () 个不同的码字。
- A. 107 B. 108 C. 214 D. 215
06. 设某哈夫曼树有 5 个叶结点，则该哈夫曼树的高度最高可以是 ()。
- A. 3 B. 4 C. 5 D. 6
07. 以下对于哈夫曼树的说法中，错误的是 ()
- A. 用一组权值构造出的哈夫曼树可能不唯一，但带权路径长度唯一
 B. 哈夫曼树具有最小的带权路径长度
 C. 哈夫曼树中没有度为 1 的结点
 D. 哈夫曼树中除了度为 1 的结点，还有度为 2 的结点和叶结点
08. 下列关于哈夫曼树的说法中，错误的是 ()。
- I. 哈夫曼树的总结点数不能是偶数
 II. 哈夫曼树中度为 1 的结点数等于度为 2 和 0 的结点数之差
 III. 哈夫曼树的带权路径长度等于其所有分支结点的权值之和
- A. 仅 III B. I 和 II C. 仅 II D. I、II 和 III

- TRAE AI 生成

19. 【2019 统考真题】对 n 个互不相同的符号进行哈夫曼编码。若生成的哈夫曼树共有 115 个结点, 则 n 的值是 ()。
- A. 56 B. 57 C. 58 D. 60
20. 【2021 统考真题】若某二叉树有 5 个叶结点, 其权值分别为 10, 12, 16, 21, 30, 则其最小的带权路径长度 (WPL) 是 ()。
- A. 89 B. 200 C. 208 D. 289
21. 【2022 统考真题】对任意给定的含 n ($n > 2$) 个字符的有限集 S , 用二叉树表示 S 的哈夫曼编码集和定长编码集, 分别得到二叉树 T_1 和 T_2 。下列叙述中, 正确的是 ()。
- A. T_1 与 T_2 的结点数相同 B. T_1 的高度大于 T_2 的高度
- C. 出现频次不同的字符在 T_1 中处于不同的层
- D. 出现频次不同的字符在 T_2 中处于相同的层
22. 【2023 统考真题】在由 6 个字符组成的字符集 S 中, 各字符出现的频次分别为 3, 4, 5, 6, 8, 10, 为 S 构造的哈夫曼编码的加权平均长度为 ()。
- A. 2.4 B. 2.5 C. 2.67 D. 2.75
23. 【2025 统考真题】设字符集 S 中包含 7 个字符, 各字符出现的频次分别为 2, 3, 4, 6, 8, 10, 11。现为 S 中的各字符构造哈夫曼编码, 编码长度不小于 3 的字符数是 ()。
- A. 2 B. 3 C. 4 D. 5

二、综合应用题

01. 设给定权集 $w = \{5, 7, 2, 3, 6, 8, 9\}$, 试构造关于 w 的一棵哈夫曼树, 并求其加权路径长度 WPL。
02. 【2012 统考真题】设有 6 个有序表 A, B, C, D, E, F, 分别含有 10, 35, 40, 50, 60 和 200 个数据元素, 各表中的元素按升序排列。要求通过 5 次两两合并, 将 6 个表最终合并为 1 个升序表, 并使最坏情况下比较的总次数达到最小。请回答下列问题:
- 1) 给出完整的合并过程, 并求出最坏情况下比较的总次数。
- 2) 根据你的合并过程, 描述 n ($n \geq 2$) 个不等长升序表的合并策略, 并说明理由。
03. 【2020 统考真题】若任意一个字符的编码都不是其他字符编码的前缀, 则称这种编码具有前缀特性。现有某字符集 (字符个数 ≥ 2) 的不等长编码, 每个字符的编码均为二进制的 0、1 序列, 最长为 L 位, 且具有前缀特性。请回答下列问题:
- 1) 哪种数据结构适宜保存上述具有前缀特性的不等长编码?
- 2) 基于你所设计的数据结构, 简述从 0/1 串到字符串的译码过程。
- 3) 简述判定某字符集的不等长编码是否具有前缀特性的过程。

5.5.4 答案与解析

一、单项选择题

01. A

由哈夫曼树的构造过程可知, 哈夫曼树中只有度为 0 和 2 的结点。在非空二叉树中, 有 $n_0 = n_2 + 1$, 所以 $n_2 = n - 1$ 。

【另解】 n 个结点构造哈夫曼树需要 $n - 1$ 次合并过程, 每次合并新建一个分支结点, 所以选择选项 A。

02. C

首先, 3 和 5 构造为一棵子树, 其根权值为 8, 然后该子树与 6 构造为一棵新子树, 根权值

为 14, 再后 9 与 12 构造为一棵子树, 最后两棵子树共同构造为一棵哈夫曼树。

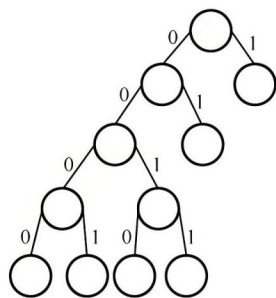
03. B

若没有一个编码是另一个编码的前缀, 则称这样的编码为前缀编码。在选项 B 中, 0 是 00 的前缀, 1 是 11 的前缀。

04. C

在哈夫曼编码中, 一个编码不能是任何其他编码的前缀。3 位编码可能是 001, 对应的 4 位编码只能是 0000 和 0001。3 位编码也可能是 000, 对应的 4 位编码只能是 0010 和 0011。若全采用 4 位编码, 则可以为 0000, 0001, 0010 和 0011。题中问的是最多, 所以选择选项 C。

【另解】若哈夫曼编码的长度只允许小于或等于 4, 则哈夫曼树的高度最高是 5, 已知一个字符编码为 1, 另一个字符编码为 01, 这说明第二层和第三层各有一个叶结点, 为使得该树从第 3 层起能够对尽可能多的字符编码, 余下的二叉树应该是满二叉树, 如右图所示, 底层可以有 4 个叶结点, 最多可以再对 4 个字符编码。



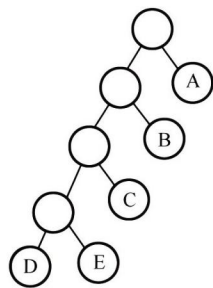
05. B

根据上题的结论, 叶结点数为 $(215 + 1)/2 = 108$, 所以共有 108 个不同的码字。

【另解】在哈夫曼树中只有度为 0 和 2 的结点, 结点总数 $n = n_0 + n_2$, 且 $n_0 = n_2 + 1$, 由题知 $n = 215$, $n_0 = 108$ 。

06. C

在哈夫曼树的构造中, 每个初始结点最终都成为叶结点, 5 个初始结点构造的哈夫曼树共新建 4 个双分支结点, 4 个双分支结点所构成的高度最高的哈夫曼树如右图所示, 其高度为 5。



07. D

在哈夫曼树的构造过程中, 每次选根的权值最小的两棵树, 一棵作为左子树, 一棵作为右子树, 生成新的二叉树, 新的二叉树根的权值应为其左右两棵子树根结点权值的和。至于谁做左子树, 谁做右子树, 没有限制, 所以构造的哈夫曼树是不唯一的, 但其带权路径长度是最小的和唯一的。哈夫曼树只有度为 0 和 2 的结点, 度为 0 的结点是外结点, 带有权值, 没有度为 1 的结点。

08. C

由 n 个初始结点构造的哈夫曼树, 会新建 $n - 1$ 个双分支结点, 因此总结点数为 $2n - 1$, 必为奇数, 说法 I 正确。哈夫曼树中没有度为 1 的结点, 说法 II 错误。哈夫曼的带权路径长度有两种计算方法: ① 所有叶结点的带权路径长度之和; ② 所有分支结点的权值之和, 说法 III 正确。

09. C

一棵度为 m 的哈夫曼树应只有度为 0 和 m 的结点, 设度为 m 的结点有 n_m 个, 度为 0 的结点有 n_0 个, 又设结点总数为 N , $N = n_0 + n_m$ 。因 N 个结点的哈夫曼树有 $N - 1$ 条分支, 则 $mn_m = N - 1 = n_m + n_0 - 1$, 整理得 $(m - 1)n_m = n_0 - 1$, $n_m = (n_0 - 1)/(m - 1)$ 。

10. B

并查集的存储结构是用双亲表示法存储的树, 主要是为了方便两个重要的操作。

11. C

初始时, 0~9 各自成一个集合。查找 1-2 时, 合并{1}和{2}; 查找 3-4 时, 合并{3}和{4};